

CopyLane 시스템 구성도

SKN Final Project 최종 산출물 — 시스템 구성도 (2026-10-21 개정)

팀장 **오한빈**

팀원 **박수진 · 권소라 · 소성민 · 이서은**

정리 **권소라 · 인코더 재료 박수진**

개정 **2026-10-21 · 기준 커밋 ksr b54246d**

제출용

목차

1. 이 문서가 담은 도면 3

2. 전체 구성 (S-01) 4

2-1. 층별로 보면 4

2-2. 이 구성이 지키는 경계 5

3. 구성요소 표 6

4. 요청 흐름 9

4-1. 문구 검수 (S-02) 9

4-2. 카피 생성 · AI 광고 생성 (S-03) 10

4-3. 고쳐 쓰기 (S-04) 11

5. 데이터 파이프라인 (S-05) 12

6. 배포 구성 (S-06) 14

6-1. 실제로 올린 것과 계획을 나눠 적는다 14

6-2. 포트 · 주소 15

6-3. 설정 키 (값은 적지 않는다) 15

7. 아직 정하지 않았거나 문서끼리 다른 것 17

부록 A. 근거표 18

1. 이 문서가 담은 도면

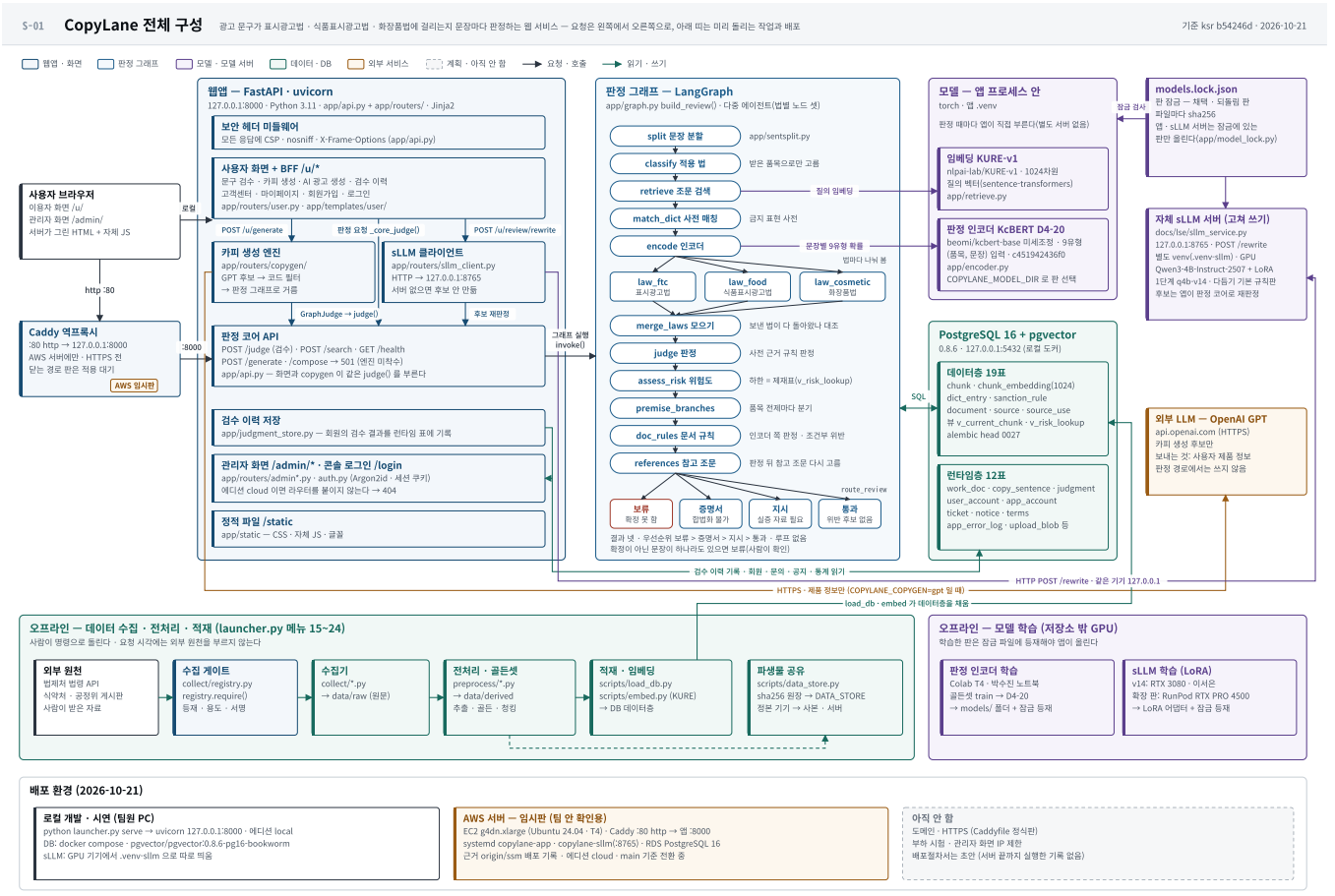
> **작성** 2026-10-21 (새로 만든 산출물) · 정리 권소라 · 인코더 부분 재료 박수진(`docs/psj/reports/시스템구성도_판정인코더_박수진_20261019.md`)
>
> **개정** 2026-10-21 · **기준 커밋** `ksr b54246d` (origin/main `2f1e86d` 포함) · 배포 사실은 `origin/ssm 9372589` 의 배포 기록에서 읽음
>
> **읽는 법** 그림의 모든 상자와 화살표는 코드나 배포 기록에 근거가 있다. 근거는 부록 A 「근거표」에 번호(E-nn)로 적었다. 실제로 도는 것, AWS 서버에 임시로 올린 것, 계획만 있는 것을 다른 모양으로 그렸다.

번호	도면	무엇을 보여 주나
S-01	전체 구성	브라우저부터 웹앱 · 판정 그래프 · 모델 · DB · 외부 서비스 · 오프라인 파이프라인 · 배포 환경까지 한 장
S-02	요청 흐름 ① 문구 검수	`/u/review` 요청 하나가 판정 그래프 노드를 지나 화면과 이력 저장으로 돌아오는 길
S-03	요청 흐름 ② 카피 생성 · AI 광고 생성	GPT 후보 → 코드 필터 → 판정 그래프 반복, 그리고 AI 광고 생성 편집기의 판정
S-04	요청 흐름 ③ 고쳐 쓰기	검수 화면에서 자체 sLLM 서버로 가서 후보를 받고 다시 판정하는 길
S-05	데이터 파이프라인	외부 원천 → 수집 게이트 → 원문 → 전처리 → 적재 · 임베딩, 그리고 파생물 공유
S-06	배포 구성	로컬 개발 · 시연 구성, AWS 임시판 구성, 아직 하지 않은 것

범례(모든 도면 공통)

모양	뜻
남색 테두리	웹앱(FastAPI) 안의 화면 · BFF · API
파란 둥근 상자	판정 그래프(LangGraph) 노드
보라	모델(판정 인코더 · 임베딩)과 자체 sLLM 서버 · 모델 잠금
초록	데이터 · PostgreSQL · 파생물
주황	외부 서비스(OpenAI) · AWS 임시판
회색 점선	계획만 있거나 아직 하지 않은 것 · 엔진 미착수(501)
검은 화살표	요청 · 함수 호출
초록 화살표	DB 읽기 · 쓰기

2. 전체 구성 (S-01)



A4 에서는 글자가 작다 — PDF 를 확대하면 선명하게 보이고(벡터), 같은 그림을 큰 PNG 로 따로 냈다(`13_시스템_구성도_전체_2026-10-21.png`).

한 줄 요약. 사용자는 브라우저로 FastAPI 웹앱 하나에 붙는다. 화면(BFF)은 판정 코어 함수 `judge()` 를 불러 LangGraph 판정 그래프를 돌리고, 그래프는 PostgreSQL + pgvector 의 법령 청크 · 사전 · 제재표와 앱 프로세스 안의 판정 인코더 · 임베딩 모델을 읽는다. 카피 생성만 외부 GPT 를 부르고, 고쳐 쓰기는 같은 기기의 자체 sLLM 서버(8765)를 부른다. 판정 근거 데이터는 요청 때가 아니라 오프라인 파이프라인에서 미리 만들어 DB 에 넣는다.

2-1. 층별로 보면

층	무엇	상태 (2026-10-21)
사용자	브라우저 — 이용자 화면 `/u/` , 관리자 화면 `/admin/` . 화면은 서버가 Jinja2 로 그린 HTML 과 자체 JS(`app/static`)	동작 · 화면은 2026-10-21 기준 화면이며 확정이 아니다
웹앱	FastAPI 앱 하나(`app/api.py` + `app/routers/`). uvicorn 으로 `127.0.0.1:8000`	동작
판정 그래프	LangGraph `build_review()` — 공통 노드와 법별 판정 노드 셋(표시광고법 · 식품표시광고법 · 화장품법). 루프 없음	동작
모델(앱 안)	판정 인코더 KcBERT D4-20(9유형 다중 라벨) · 임베딩 KURE-v1(1024차원). 별도 서버 없이 앱 프로세스 안에서 올린다	동작 · 판은 `models.lock.json` 잠금에 있는 것만

층	무엇	상태 (2026-10-21)
데이터	PostgreSQL 16 + pgvector 0.8.6. 데이터층 19표(법령 청크 · 임베딩 · 사전 · 제재표 · 골든셋 등) + 런타임층 12표(이력 · 회원 · 문의 등). alembic head `0027`	동작
별도 프로세스	자체 sLLM 서버 `docs/lse/sllm_service.py` — `127.0.0.1:8765` · GPU 용 가상환경 `.venv-sllm`	동작(GPU 기기에서 따로 띄움)
외부 서비스	OpenAI API — 카피 생성 후보만. 판정 경로에서는 쓰지 않는다	동작 · `.env` 스위치 `COPYLANE_COPYGEN=gpt` 일 때만
오프라인	데이터 수집 · 전처리 · 적재(`launcher.py` 메뉴 15~24) · 모델 학습(저장소 밖 GPU)	사람이 명령으로 돌림
배포	로컬(팀원 PC) · AWS g4dn 임시판(IP · http · 팀 안 확인용)	임시판까지. 도메인 · HTTPS · 부하 시험은 아직

2-2. 이 구성이 지키는 경계

1. **판정 코어는 하나다.** 검수 화면(`/u/review`), 품목 · 전제를 고른 뒤 다시 판정, 분기 질문 답, 고쳐 쓴 후보의 재판정, 카피 생성 후보 거르기, AI 광고 생성 편집기 판정이 모두 `app/api.py` 의 같은 `judge()` 함수를 부른다. 화면마다 판정 로직을 따로 두지 않는다.
2. **외부로 나가는 것은 카피 생성 한 곳뿐이다.** `app/routers/copygen/llm.py` 만 `api.openai.com` 을 부르고, 보내는 것은 사용자가 입력한 제품 정보다. 인정 · 심사 문구, 검수 문구, 판정 결과는 보내지 않는다. 판정 경로(검수 · 재판정 · 인코더)에서는 GPT 를 쓰지 않는다(D-332).
3. **추적 도구로 문구가 새지 않게 막는다.** LangSmith 추적이 켜져 있으면 그래프를 짓지 않고 멈춘다(`app/graph.py` `_require_tracing_off`).
4. **모델은 잠금에 있는 판만 올린다.** 판정 인코더와 sLLM 어댑터 모두 `models.lock.json` 의 채택 · 되돌림 판인지 sha256 으로 대조한 뒤 올린다.
5. **없는 기능은 없다고 낸다.** 코어 `POST /generate` · `POST /compose` 는 엔진이 없어 501 을 낸다. sLLM 서버가 없으면 고쳐 쓰기 후보를 지어내지 않는다. DB 나 그래프 의존성이 없으면 503 을 내고 화면은 「판정 엔진에 연결하지 못했어요」로 따로 안내한다.
6. **안쪽 포트는 밖에 열지 않는다.** 앱 8000 · sLLM 8765 · DB 5432 는 모두 `127.0.0.1` 에만 묶는다. AWS 서버에서는 Caddy 가 80 번만 받아 앱으로 넘긴다.

3. 구성요소 표

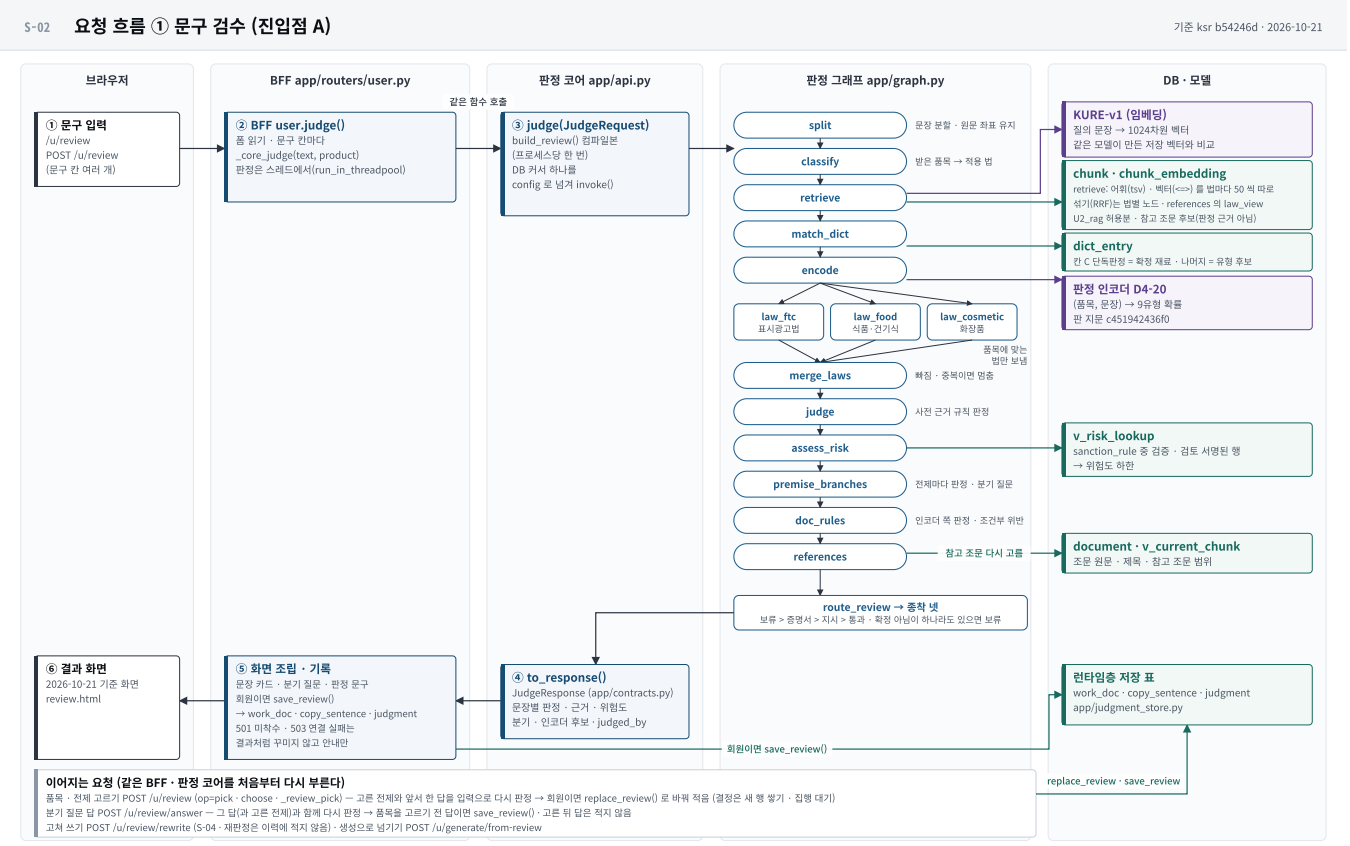
#	이름	역할	기술	포트 · 주소	코드 위치	상태
1	사용자 브라우저	이용자 · 관리자 화면 사용	HTML(Jinja2) + 자체 JS	—	`app/templates/` · `app/static/`	동작
2	보안 헤더 미들웨어	모든 응답에 CSP · nosniff · X-Frame-Options 등	FastAPI middleware	—	`app/api.py:106-120`	동작
3	사용자 화면 + BFF	문구 검수 · 카피 생성 · AI 광고 생성 · 검수 이력 · 고객센터 · 마이페이지 · 가입 · 로그인	FastAPI router · Jinja2	`/u/*`	`app/routers/user.py`	동작(2026-10-21 기준 화면)
4	판정 코어 API	검수 판정 · 조문 검색 · 상태 확인	FastAPI	`POST /judge` · `POST /search` · `GET /health`	`app/api.py`	동작
5	생성 · 조립 코어 API	계약만 있고 엔진 없음	FastAPI	`POST /generate` · `POST /compose` → 501	`app/api.py:533-554`	미착수
6	카피 생성 엔진	GPT 후보 → 코드 필터 → 판정 그래프로 거름 → A안 · B안 · C안	Python · 표준 `urllib`	`POST /u/generate` 에서 호출	`app/routers/copygen/`	동작(스위치 켤 때)
7	sLLM 클라이언트	고쳐 쓰기 요청 · 후보 재판정 연결	표준 `urllib`	→ `127.0.0.1:8765`	`app/routers/sllm_client.py`	동작
8	검수 이력 저장	회원의 검수 결과를 런타임 표에 기록	SQLAlchemy	—	`app/judgment_store.py` (`save_review` · `replace_review`)	동작 · 품목을 고른 재판정은 지금 코드가 앞 기록을 지우고 바꿔 적는다. 결정기록은 「새 행으로 쌓는다」로 정했고(D-348 보충 36 결정 2) 집행 대기
9	관리자 화면 · 콘솔 로그인	회원 · 문의 · 공지 · 약관 · 통계 · 오류 · 소스 관리	FastAPI router · Argon2id · 세션 쿠키	`/admin/*` · `/login`	`app/routers/admin*.py` · `app/routers/auth.py` · `app/auth.py`	동작 · 에디션 `cloud` 에서는 라우터를 붙이지 않음(404)
10	판정 그래프	문장 분할 → 법 고르기 → 조문 검색(참고 조문 후보) → 사전 매칭 → 인코더 → 법별 판정 셋 → 모으기 → 판정 → 위험도 → 품목 분기 → 문서 규칙 → 참고 조문 → 종착 넷	LangGraph(≥ 1.2.11)	—	`app/graph.py` (`CORE_BEFORE_LAWS` · `CORE_AFTER_LAWS` · `build_review`)	동작

#	이름	역할	기술	포트 · 주소	코드 위치	상태
11	조문 검색	어휘(tsvector) · 벡터(코사인) 두 갈래. 판정 그래프에서는 `wide` 가 두 갈래를 법마다 따로 받고, 법별 노드 · 참고 조문 노드가 `law_view` 로 자기 법 것만 골라 RRF 로 섞는다(`POST /search` 는 `search` 가 바로 섞음) · 용도 U2(RAG)가 허용된 원천만 · 결과는 참고 조문 재료이고 판정 근거가 아니다	SQL · pgvector `<=>`	—	`app/retrieve.py`	동작 · 리랭커 없음
12	판정 인코더	(품목, 문장) → 9유형 확률	KcBERT 미세조정 D4-20(`beomi/kcbert-base`) · torch	앱 프로세스 안	`app/encoder.py` · `models/<판 이름>` · `.env COPYLANE_MODEL_DIR`	동작 · 판 지문 `c451942436f0`
13	임베딩	질의 문장 → 1024차원 벡터	nlpai-lab/KURE-v1 · sentence-transformers	앱 프로세스 안	`app/retrieve.py` · `scripts/embed.py`	동작
14	모델 잠금	채택 · 되돌림 판과 파일별 sha256	JSON	—	`models.lock.json` · `app/model_lock.py`	동작
15	PostgreSQL + pgvector	데이터층 19표 · 런타임층 12표 · 뷰	PostgreSQL 16 · pgvector 0.8.6 · alembic	`127.0.0.1:5432`	`db/schema.sql` · `alembic/` · `app/models.py` · `docker-compose.yml`	동작 · head `0027`
16	자체 sLLM 서버	지적 문장 고쳐 쓰기(1단계 → 후처리 · 관문 → 2단계 다듬기)	Qwen3-4B-Instruct-2507 + LoRA(PEFT) · FastAPI · GPU torch	`127.0.0.1:8765` `POST /rewrite` · `GET /health`	`docs/lse/sllm_service.py` · `docs/lse/sllm_meta.py`	동작 · 1단계 `q4b-v14` · 2단계 다듬기는 코드 기본값이 규칙판 · 어댑터 `q4b_polish` (잠금 채택)는 고객층 말투 · 모델판 다듬기용
17	외부 LLM	카피 생성 후보	OpenAI Chat Completions · 기본 모델 `gpt-5.6-luna`, `gpt-6-luna`	`api.openai.com` 443(HTTPS)	`app/routers/copygen/llm.py`	동작(스위치 · 키 · 인코더가 다 있을 때)
18	수집 게이트 · 수집기	등재 · 용도 · 서명을 본 뒤에만 수집	Python	—	`collect/registry.py` · `collect/*.py` · `data_sources.yaml`	오프라인
19	전처리 · 골든셋 · 청킹	원문 → 파생물	Python	—	`preprocess/*.py`	오프라인

#	이름	역할	기술	포트 · 주소	코드 위치	상태
20	적재 · 임베딩	파생물 → DB 데이터층	Python · psycopg	—	`scripts/load_db.py` · `scripts/embed.py`	오프라인
21	파생물 공유	정본 기기가 올리고 사본 기기 · 서버가 받음 (sha256 대조)	폴더 저장소	`.env DATA_STORE`	`scripts/data_store.py` · `data/derived_manifest.jsonl`	오프라인
22	런처	환경 · DB · 수집 · 전처리 · 서버 실행을 한 메뉴로	typer	`serve` 기본 `127.0.0.1:8000`	`launcher.py`	동작 · `train` · `demo` 는 미구현
23	Caddy 역프록시	외부 80 → 앱 8000 · `/` → `/u/` . 저장소의 `Caddyfile.ip` 는 개발용 경로와 관리자 경로를 404 로 닫고 본문을 1MB 로 묶는다	Caddy	`:80` (임시판)	`docs/ssm/배포/Caddyfile.ip` (origin/ssm)	AWS 임시판 — 서버의 지금 판은 닫는 경로가 없는 10-16 임시 구성이고 `Caddyfile.ip` 적용은 `main-switch` 대기 · HTTPS 정식판은 아직
24	배포 스크립트	서버 준비 · 설치 · 적재 · 지문 대조 · systemd 서비스 · Caddy 적용	bash · systemd	—	`docs/ssm/배포/deploy_server.sh` (origin/ssm)	AWS 임시판

4. 요청 흐름

4-1. 문구 검수 (S-02)



1. 사용자가 `/u/review` 에서 문구 칸을 채우고 보낸다(`POST /u/review`).
2. BFF(`app/routers/user.py` 의 `judge` — `POST /u/review` · `POST /u/judge`)가 문구 칸마다 `_core_judge()` 를 부른다. 판정은 이벤트 루프를 막지 않게 스레드로 넘긴다(`run_in_threadpool`).
3. `_core_judge()` 는 `app/api.py` 의 `judge()` 를 그대로 부른다(`POST /judge` 와 같은 함수). `judge()` 는 프로세스당 한 번 컴파일한 검수 그래프에 DB 커서 하나를 `config` 로 넘겨 돌린다.
4. 그래프 코어 노드가 차례로 돈다.
   - `retrieve` 는 KURE-v1 로 질의 벡터를 만들어 벡터 갈래(`chunk_embedding`)와 어휘 갈래를 법 필터 없이 한 번, 법마다 50 개씩 **따로** 받는다(`rt.wide` — 섞지 않는다). 받은 것은 참고 조문 후보이고 판정 근거가 아니다.
   - `match_dict` 는 금지 표현 사전(`dict_entry`)을 훑는다. 단독판정 항목 중 불가 사유 칸이 C 인 것만 확정 재료로 내고, 나머지 항목의 적중은 보류 문장의 유형 후보로 낸다.
   - `encode` 는 판정 인코더로 문장마다 9유형 확률을 낸다(법별 노드로 나뉘기 전에 한 번).
   - 법별 노드 셋은 받은 품목에 맞는 법만 받아(품목을 모르면 셋 다) 두 갈래 후보에서 자기 법 것만 골라 RRF 로 섞고 (`rt.law_view`) 자기 법을 인용한 사전 적중만 거른다. 판정은 하지 않는다.
   - `judge` 가 사전 쪽 판정을 내고, `assess_risk` 는 서명이 끝난 제재표(`v_risk_lookup`)로 위험도 하한을 건다.
   - `premise_branches` 는 품목 전제마다 판정을 미리 계산한다(사용자가 전제를 골라 넣었으면 그 전제 하나로 판정하고 분기는 없다). `doc_rules` 는 인코더 쪽 판정과 조건부 위반(사실 · 실증 질문)을 단다. 분기가 있고 품목을 모르면 기록 문

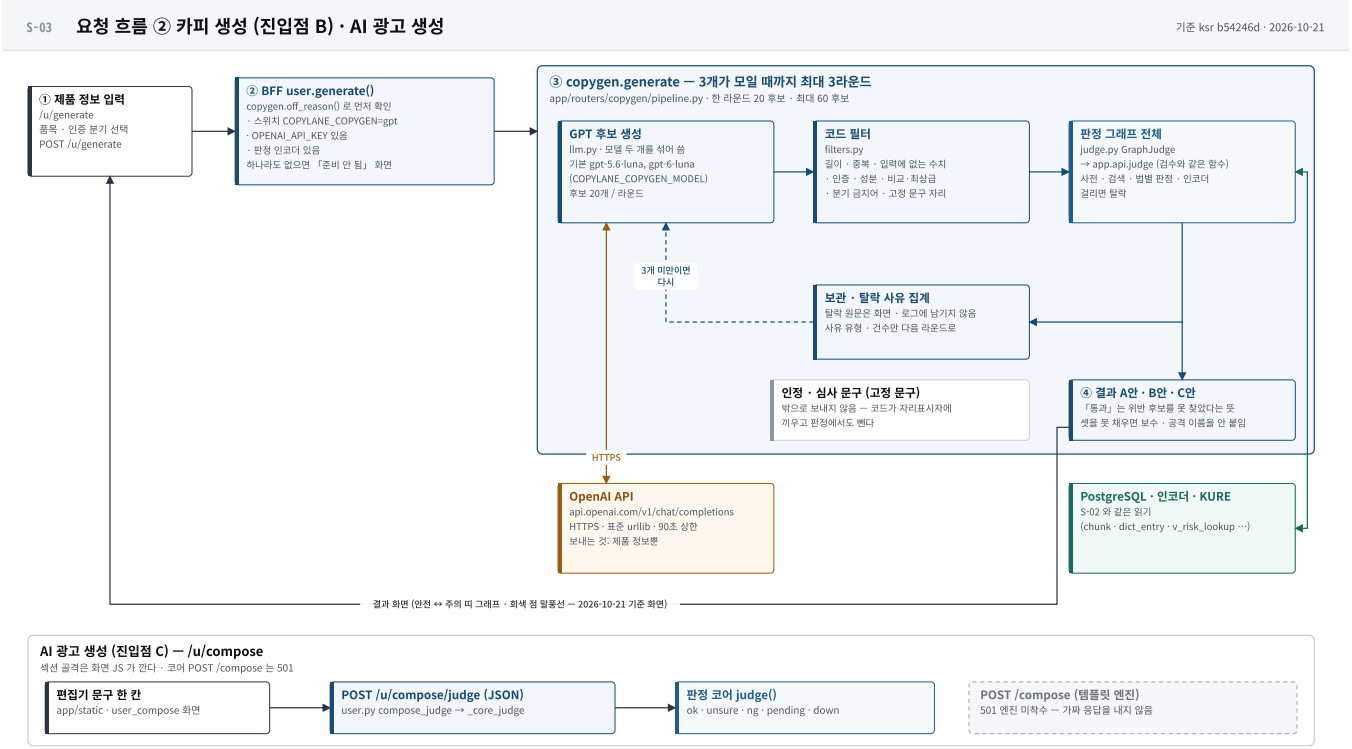
장의 질문은 걷어 「고르기 대기」(`cat_unknown`)로 두고, 질문은 분기 미리보기 안에만 남긴다(`_pick_first` · D-348 보충 37 결정 2).

- `references` 는 판정이 정해진 뒤 같은 후보에서 (유형 · 법) 안의 참고 조문을 다시 고른다(`rt.law_view` 로 섞음).

5. `route_review` 가 종착 넷 중 하나로 보낸다. 우선순위는 보류 > 증명서 > 지시 > 통과이고, 확정이 아닌 문장이 하나라도 있으면 보류다. 루프는 없다.
6. `to_response()` 가 계약(`app/contracts.py` `JudgeResponse`)대로 응답을 만든다. 보류 문장에는 보류의 종류(`hold_kind` — 답 대기 · 고르기 대기 · 확인 못 함 · 걸린 것 없음 · 그 밖)가 계산 칸으로 함께 실린다(D-348 보충 36 결정 4). 판정을 다 낸 뒤에는 고르는 칸(품목 · 전제 선택지 · 실증 · 사실 질문)마다 승인된 두 겹 안내(쉬운 풀이 · 원문 보기)가 붙는다(`app/guide.py` `attach` · 보충 37 결정 7 · 판정에는 들지 않는다). 두 칸 모두 2026-10-21 기준 화면은 아직 그리지 않는다. BFF 는 문장 카드 · 분기 질문을 그리고, 회원이면 `save_review()` 로 `work_doc` · `copy_sentence` · `judgment` 에 기록한다.
7. 이어지는 요청(모두 같은 판정 코어를 처음부터 다시 부른다 — 그래프 안의 루프가 아니다):
   - 품목 · 전제 고르기(`POST /u/review` 의 `op=pick` · `choose` — `_review_pick`): 고른 전제와 앞서 한 답을 입력으로 다시 판정하고, 회원이면 `replace_review()` 로 그 문구의 앞 기록을 지우고 바꿔 적는다. 결정기록은 「새 행으로 쌓는다」로 정했고 집행 대기다(D-348 보충 36 결정 2).
   - 분기 질문 답(`POST /u/review/answer`): 그 답(과 고른 전제)과 함께 다시 판정한다. 품목을 고르기 전의 답이면 `save_review()` 로 적고, 고른 뒤의 답은 적지 않는다.
   - 고쳐 쓰기(`POST /u/review/rewrite` — 4-3 · 재판정 결과는 이력에 적지 않는다), 생성으로 넘기기(`POST /u/generate/from-review`).

엔진이 없으면 501(미착수), DB · 그래프 의존성이 없으면 503(연결 실패)이고, 화면은 둘을 결과처럼 꾸미지 않고 다른 안내로 그린다.

4-2. 카피 생성 · AI 광고 생성 (S-03)



1. 사용자가 `/u/generate` 에서 품목 · 인증 분기를 고르고 제품 정보를 넣는다.

2. BFF 는 먼저 `copygen.off_reason()` 으로 세 가지를 본다 — `.env` 스위치 `COPYLANE_COPYGEN=gpt`, `OPENAI_API_KEY`, 판정 인코더. 하나라도 없으면 생성하지 않고 「준비 안 됨」을 그린다. 키가 있다는 것만으로는 켜지지 않는다.
3. `copygen.generate` 가 한 라운드에 후보 20개를 GPT 에 요청한다(모델 둘을 섞어 10개씩). 후보는 코드 필터(길이 · 중복 · 입력에 없는 수치 · 인증 · 성분 · 비교 · 최상급 · 분기 금지어 · 고정 문구 자리)를 지난다.
4. 남은 후보는 `GraphJudge` 가 검수와 같은 `app.api.judge` 로 판정 그래프 전체에 넣고, 걸리면 탈락시킨다.
5. 세 개가 모일 때까지 최대 3라운드(최대 60후보) 반복한다. 탈락 문구 원문은 화면 · 로그에 남기지 않고 사유 유형과 건수만 다음 라운드에 알린다.
6. 결과는 A안 · B안 · C안으로 보인다. 「통과」는 위반 후보를 찾지 못했다는 뜻이지 적법 확인이 아니다.

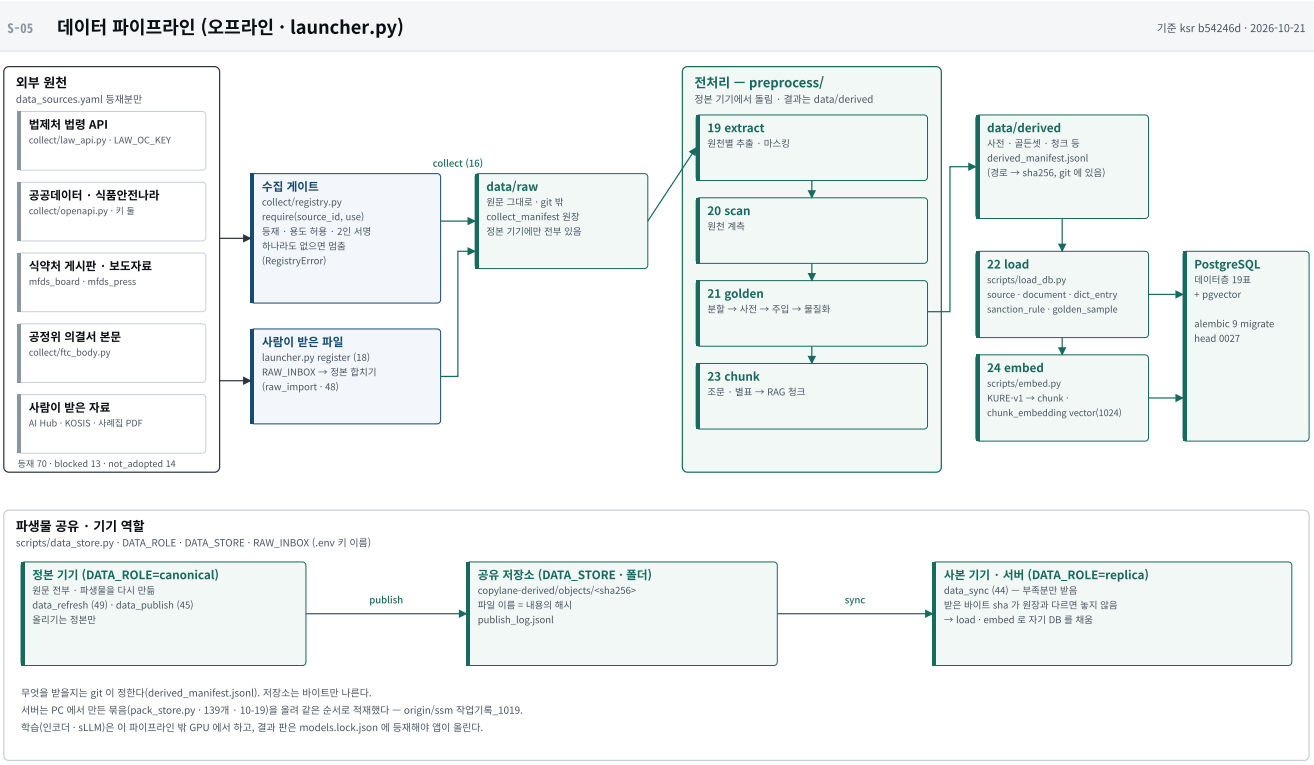
AI 광고 생성(`/u/compose`). 섹션 골격은 화면 JS 가 깔고, 편집기에서 문구 한 칸을 고치면 `POST /u/compose/judge` 가 같은 판정 코어로 판정한다(`ok` · `unsure` · `ng` · `pending` · `down`). 코어의 템플릿 엔진 `POST /compose` 는 501 이다.

4-3. 고쳐 쓰기 (S-04)



1. 검수 결과의 지적 문장에서 「고쳐 쓰기」를 누르면 `POST /u/review/rewrite` 가 문구 칸 · 대상 문장 · 고른 전제를 보낸다.
2. BFF 는 문구를 **다시 판정**하고 위반 유형을 서버의 판정 결과에서 읽는다(폼 값을 믿지 않는다). 품목 분기가 있는 문구는 전제를 먼저 고르게 하고, 고른 전제 · 앞서 한 답이 있으면 그것을 실어 다시 판정한 결과에서 읽는다.
3. `sllm_client.rewrite()` 가 `127.0.0.1:8765/rewrite` 에 문장 · 위반 유형 · 품목을 보낸다(대기 상한 90초).
4. sLLM 서버는 1단계 어댑터(`q4b-v14`)로 위반 표현을 지운 문장을 만들고, 후처리 · 관문을 거친 뒤 2단계 다듬기를 한다. 다듬기는 코드 기본값이 규칙판이다(`docs/lse/persona_pipeline_e2e.py:90` `POLISH_MODE = "rule"`). 응답은 `candidate` · `hold` · `infeasible` 중 하나와 사유다.
5. BFF 는 받은 후보마다 판정 코어로 다시 판정하고(`_rejudge` — 고른 전제가 있으면 그 전제를 입력으로), 다듬은 안이 걸리면 1단계 담백한 안을 보인다. `_rejudge` 는 앞서 한 답을 받을 칸이 있지만(D-348 보충 37 결정 5) 화면 경로(`_rewrite_sentence`)는 아직 답을 넘기지 않는다 — 고친 문장의 재판정에는 고른 전제의 제품 정보만 실린다.
6. `COPYLANE_SLLM_URL=off` (또는 빈 값)면 고쳐 쓰기를 두지 않은 환경으로 보고 버튼 대신 안내를 그린다. 서버가 없거나 응답이 깨지면 후보를 지어내지 않는다.

5. 데이터 파이프라인 (S-05)

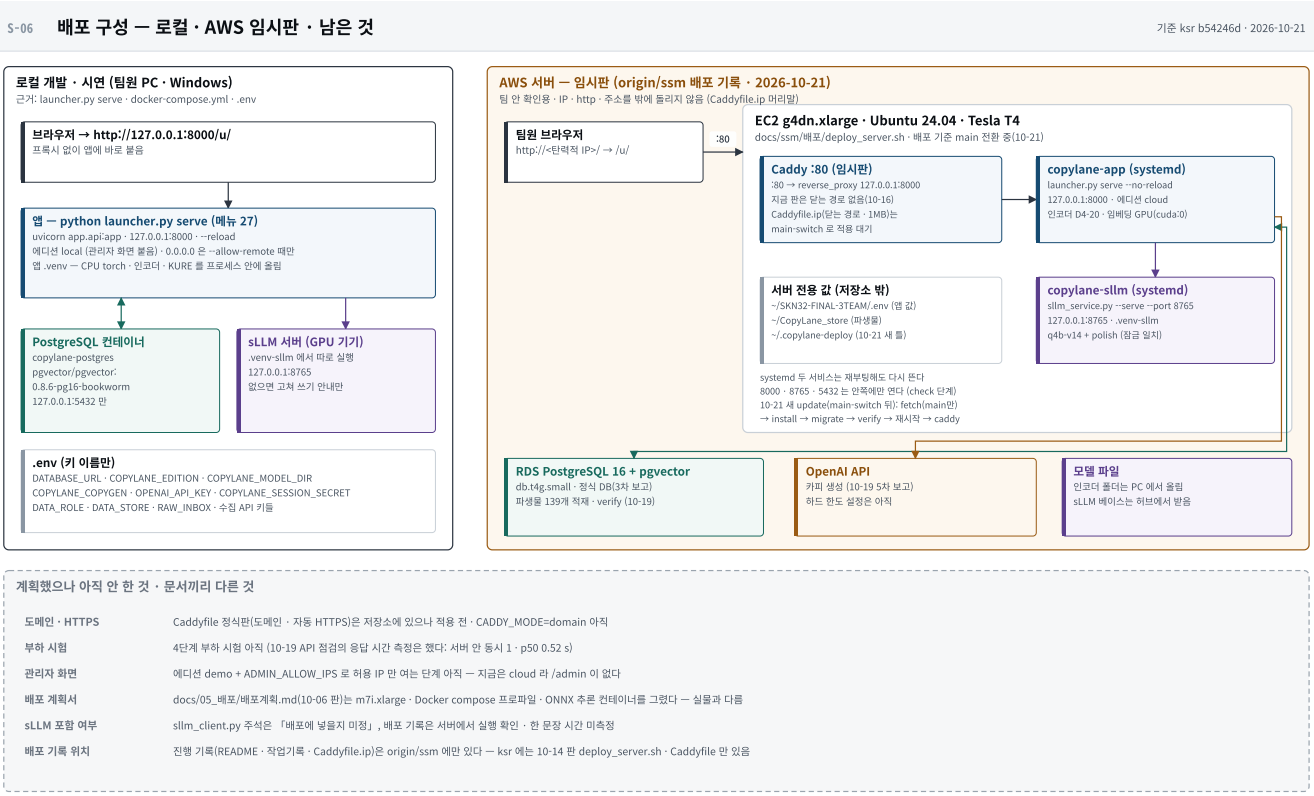


단계	명령(런처 메뉴)	코드	결과
수집 게이트	—	`collect/registry.py` `require(source_id, use)`	등재 · 용도 허용 · 2인 서명이 다 있어야 수집이 시작된다. 없으면 `RegistryError` 로 멈춘다
오픈 API · 게시판 수집	16 collect	`collect/law_api.py` · `openapi.py` · `mfds_board.py` · `mfds_press.py` · `ftc_body.py` 등	`data/raw` 원문 · `collect_manifest`
사람이 받은 자료	18 register · 47/48 원문 올리기 · 합치기	`collect/__init__.py` `MANUAL_SOURCES` · `scripts/raw_inbox.py`	AI Hub · KOSIS · 사례집 PDF 등을 정본 기기로 모음(`.env` `RAW_INBOX`)
전처리 추출 · 계측	19 extract · 20 scan	`preprocess/*.py`	`data/derived`
골든셋	21 golden	`preprocess.split` → `dictionary` → `inject` → `golden`	골든셋 · 사전 파생물
적재	22 load	`scripts/load_db.py`	source · source_use · source_constraint · fragment · document · collect_manifest · dict_entry · product_fact · violation_article · sanction_rule · golden_sample
청킹	23 chunk	`preprocess/chunk.py`	조문 · 별표 → RAG 청크
임베딩	24 embed	`scripts/embed.py`	KURE-v1 로 `chunk` · `chunk_embedding vector(1024)` 적재

단계	명령(런처 메뉴)	코드	결과
파생물 공유	45 올리기 · 44 받기 · 49 다시 만들기	`scripts/data_store.py`	정본 기기(`DATA_ROLE=canonical`)만 올리고, 사본 기기 · 서버(`replica`)는 sha256 이 원장과 같은 것만 받는다

- 무엇을 받아야 하는지는 git 에 있는 `data/derived_manifest.jsonl` (경로 → sha256)이 정한다. 공유 저장소는 내용 해시를 이름으로 쓰는 폴더다.
- AWS 서버는 PC 에서 만든 파생물 묶음(139개)을 올려 같은 순서(migrate → 받기 → load → embed)로 채우고 지문을 대조했다(origin/ssm `docs/ssm/배포/작업기록_1019.md`).
- 2026-10-21 현재 수치(사전 · 골든셋 · 청크 수 등)는 현행 기준표 3절과 각 데이터 산출물을 따른다. 이 문서는 구조만 다룬다.
- 모델 학습은 이 파이프라인 밖의 GPU 에서 한다 — 판정 인코더 D4-20 은 Colab T4, 지금 서버에 올린 sLLM 1단계 `q4b-v14` 는 RTX 3080(이서은 PC), 확장 판(q4b-v16 · n1 · v17 · n2)은 RunPod RTX PRO 4500. 결과 판은 `models.lock.json` 에 등재해야 앱이 올린다.

6. 배포 구성 (S-06)



6-1. 실제로 올린 것과 계획을 나눠 적는다

구분	내용	근거
로컬 개발 · 시연	`python launcher.py serve` → uvicorn `127.0.0.1:8000` (에디션 `local`) · DB 는 docker compose 의 `pgvector/pgvector:0.8.6-pg16-bookworm` 컨테이너(`127.0.0.1:5432`) · sLLM 은 GPU 기기에서 따로 띄움	`launcher.py:1623-1628` · `docker-compose.yml`
AWS 임시판 (실제)	EC2 g4dn.xlarge(Ubuntu 24.04 · Tesla T4) · 탄력적 IP · Caddy `:80` http → 앱 `127.0.0.1:8000` (지금 판은 닫는 경로가 없는 10-16 임시 구성) · systemd 서비스 `copylane-app` · `copylane-sllm` (8765) · DB 는 RDS PostgreSQL 16 + pgvector(db.t4g.small) · 판정 인코더 D4-20 · 카피 생성 (OpenAI) 동작	origin/ssm `docs/ssm/배포/README.md` 「지금 어디까지 (2026-10-21)」 · `deploy_server.sh` · 작업기록_1019.md
임시판의 성격	HTTPS 전이라 **팀 안 확인용**이다. 주소를 밖에 돌리지 않는다	`Caddyfile.ip` 머리말
배포 기준	서버는 origin/main 에 들어간 커밋만 올린다(10-21 「main 하나로」 전환 진행 중)	origin/ssm README
아직 안 함	닫는 경로 · 1MB 상한을 담은 `Caddyfile.ip` 서버 적용(`main-switch` 대기) · 도메인 · HTTPS(정식 `Caddyfile`) · 4단계 부하 시험 · 관리자 화면 IP 제한 (에디션 `demo` + 허용 IP) · sLLM 한 문장 시간 측정 · OpenAI 하드 한도 확인	origin/ssm README · 작업기록_1019
계획 문서와 다른 점	`docs/05_배포/배포계획.md` (10-06 판)는 m7i.xlarge · Docker compose 프로파일 · Caddy · ONNX 추론 컨테이너 구성을 그렸다. 실제는 g4dn.xlarge 위 systemd 두 서비스 + RDS 다	`docs/05_배포/배포계획.md` §0 · §1

- 10-19 서버 API 점검(origin/ssm 작업기록_1019): `load_test.py judge` 로 서버 안 동시 1 · 5요청 p50 0.52 s, 동시 10 · 50요청 p50 2.73 s(실패 0 · uvicorn 워커 1 · sLLM 없음). 이것은 API 점검 측정이며, 배포 계획의 4단계 부하 시험은 아직이다.
- 2026-10-21 현재 배포 상태는 「AWS g4dn 서버에 IP · http 임시판으로 올려 팀 안에서 확인 중」이다. HTTPS 공개 서비스 · 도메인 운영 · 부하 시험 완료 · 상시 공개 단계는 아니다.

6-2. 포트 · 주소

포트	무엇	묶는 곳	근거
8000	웹앱(uvicorn)	`127.0.0.1` (0.0.0.0 은 `--allow-remote` 를 줄 때만)	`launcher.py:1625-1628`
8765	자체 sLLM 서버	`127.0.0.1`	`docs/lse/sllm_service.py:201` · `app/routers/sllm_client.py:20`
5432	PostgreSQL(로컬 도커)	`127.0.0.1`	`docker-compose.yml:29` · `app/settings.py:55`
80	Caddy(AWS 임시판)	서버 공인 IP	origin/ssm `Caddyfile.ip`
443	밖으로 나가는 HTTPS — OpenAI API	—	`app/routers/copygen/llm.py:21`
443(수신)	정식 HTTPS	아직 안 함	origin/ssm README

6-3. 설정 키 (값은 적지 않는다)

키	쓰는 곳	뜻
`DATABASE_URL`	`app/settings.py`	DB 접속(기본 `127.0.0.1:5432`)
`COPYLANE_EDITION`	`app/settings.py`	`local` · `demo` · `onprem` · `cloud` . `cloud` 면 관리자 화면 · 로그인 라우터를 붙이지 않는다
`COPYLANE_SESSION_SECRET`	`app/settings.py`	세션 서명
`COPYLANE_ERROR_LOG`	`app/settings.py`	오류 기록 표(`app_error_log`) 사용 여부
`COPYLANE_MODEL_DIR`	`app/encoder.py`	판정 인코더 판 이름(잠금 안에서 고름)
`COPYLANE_COPYGEN` · `COPYLANE_COPYGEN_MODEL`	`app/routers/copygen/`	카피 생성 스위치(`gpt` 일 때만 켜짐) · 모델 이름
`OPENAI_API_KEY`	`app/routers/copygen/llm.py`	카피 생성용 키
`COPYLANE_SLLM_URL`	`app/routers/sllm_client.py`	sLLM 서버 주소 · `off` 면 고쳐 쓰기 없음
`LANGCHAIN_TRACING_V2` · `LANGSMITH_TRACING`	`app/graph.py`	켜져 있으면 그래프가 멈춘다(문구 유출 방지)
`DATA_ROLE` · `DATA_STORE` · `RAW_INBOX` · `DATA_DEVICE`	`scripts/data_store.py` · `collect/store.py`	기기 역할 · 파생물 공유 폴더 · 원문 받은편지함 · 기기 별칭

키	쓰는 곳	뜻
`LAW_OC_KEY` · `DATA_GO_KR_KEY` · `FOODSAFETY_KEY` · `KOSIS_KEY` · `AIHUB_KEY`	`collect/env.py`	수집용 키(오프라인)
`POSTGRES_USER` · `POSTGRES_PASSWORD` · `POSTGRES_DB`	`docker-compose.yml`	로컬 DB 컨테이너

서버의 앱 값은 `~/SKN32-FINAL-3TEAM/.env` 에 둔다. 10-21 새 배포 스크립트는 서버 전용 값(DB 종류 · Caddy 판 · 도메인 · 관리자 허용 IP)을 저장소 밖 `~/.copylane-deploy` 로 옮긴다 — 서버에서 `main-switch` 를 돌린 뒤의 구성이다(origin/ssm `deploy.conf.example` · README).

7. 아직 정하지 않았거나 문서끼리 다른 것

항목	상태
생성 그래프 `build_generate()` (진입점 B)	앱에서 부르는 곳이 없는 스텁이다. 실제 카피 생성은 `app/routers/copygen` 이 맡는다. 이 그래프를 채울지는 판정 대기
코어 `POST /generate` · `POST /compose`	501(엔진 미착수)
리랭커	들이지 않았다(봉인 평가 뒤 안건)
법별 노드가 받는 근거 수	`top_k` 5 를 임시로 쓴다 · 판정 대기
품목 판별	`classify` 는 받은 품목만 쓴다. 그래프가 품목을 판별하는 일은 미집행
검수 이력 덮어쓰기 · 새 행 쌓기	코드에는 품목을 고른 재판정을 바꿔 적는 `replace_review` 가 있다. 결정기록은 「고른 재판정은 새 판정 행 · 출처 `user_selected` · 홈 통계는 가장 최근 판정만」으로 정했고(D-348 보충 36 결정 2) 집행 대기
고쳐 쓰기 재판정에 사용자의 답	`_rejudge` 에 칸은 생겼고(보충 37 결정 5) 화면에서 넘기는 배선은 아직
보류 종류(`hold_kind`) · 두 겹 안내 표시	판정 응답에는 실렸고(보충 36 결정 4 · 보충 37 결정 7) 화면이 그리는 일은 아직
sLLM 배포 포함	`sllm_client.py` 주석은 「배포 포함 미정」, origin/ssm 배포 기록은 서버에서 실행 확인(10-19). 승인 범위는 판정 대기로 남아 있다
sLLM 2단계 다듬기 기본값	코드 기본값은 규칙판. 이 기본값을 정한 결정(D-342)은 기록상 「집행 · 흡수 대기」
접속기록 1년 보관	표 없이 로거로만 남긴다(미구현)
배포	도메인 · HTTPS · 부하 시험 · 관리자 IP 제한 아직 · 배포절차서는 초안

부록 A. 근거표

도면의 상자 · 화살표마다 근거를 적었다. 경로 · 줄 번호는 저장소 `ksr b54246d` 기준이고, `origin/ssm:` 이 붙은 것은 배포 브랜치(`9372589`)의 파일이다.

번호	도면 · 요소	근거 (경로:줄 · 기록)
E-01	브라우저 → 웹앱 `127.0.0.1:8000` (S-01 · S-06)	`launcher.py:1623-1628` (`serve` 기본 host 127.0.0.1 · port 8000 · `--allow-remote`)
E-02	화면은 Jinja2 HTML + 자체 JS (HTMX는 쓰지 않음)	`app/templates/` · `app/templates/base.html:30` (HTMX 줄이 주석 처리됨) · `app/static/*.js`
E-03	보안 헤더 미들웨어	`app/api.py:74-120`
E-04	사용자 라우터 · 정적 파일 장착	`app/api.py:126-127`
E-05	관리자 · 로그인 라우터는 에디션이 정함 (`cloud` 면 안 붙음)	`app/api.py:136-138` · `app/settings.py:210-213, 304-310`
E-06	콘솔 로그인 · Argon2id · `app_account`	`app/routers/auth.py:110, 135` · `app/auth.py:9`
E-07	판정 코어 API `/judge` · `/search` · `/health`	`app/api.py:324, 376, 426-473`
E-08	`/generate` · `/compose` 501	`app/api.py:533-554`
E-09	화면 BFF → 판정 코어(같은 함수)	`app/routers/user.py:554-576` (`_core_judge` → `app.api.judge`)
E-10	검수 요청 · 스레드 실행 · 이력 저장	`app/routers/user.py:694-764` (`judge` · `run_in_threadpool` · `judgment_store.save_review`) · `app/judgment_store.py:1-20, 64, 210, 245` (저장 조건 · `save_review` · `replace_review`)
E-11	품목 · 전제 고르기 → 다시 판정 · 바꿔 적기 / 분기 질문 답 → 다시 판정	`app/routers/user.py:717` (`op` pick · choose → `_review_pick`) · `:1102-1187` (`_review_pick` · `replace_review`) · `:1190-1264` (`review_answer` · 고르기 전 답만 `save_review`) · 결정기록 D-348 보충 36 결정 2(새 행 · 집행 대기)
E-12	고쳐 쓰기 BFF → 다시 판정 → sLLM → 후보 재판정	`app/routers/user.py:811-868` (`_rejudge` — 고른 전제를 입력으로 · 답 칸 `facts` · `substs`) · `:895-955` (`_rewrite_sentence` — `_rejudge(…, premise, category)` 로만 부름 · 답은 넘기지 않음) · `:957-1049` (`review_rewrite`) · 결정기록 D-348 보충 37 결정 5
E-13	sLLM 클라이언트 주소 · 끄는 값 · 대기 상한	`app/routers/sllm_client.py:20, 25, 63`
E-14	sLLM 서버 `127.0.0.1:8765` · `POST /rewrite` · `GET /health` · 별도 venv	`docs/lse/sllm_service.py:1-21, 185-201, 210`
E-15	sLLM 베이스 · 잠금 대조	`docs/lse/sllm_meta.py:48, 136-157` · `models.lock.json` roles `sllm_stage1` · `sllm_stage2`
E-16	2단계 다듬기 기본값 규칙판	`docs/lse/persona_pipeline_e2e.py:90` · `docs/lse/sllm_service.py:133-148`
E-17	카피 생성 BFF · 스위치 확인	`app/routers/user.py:1605-1665` · `app/routers/copygen/__init__.py:1-21, 51-70`

번호	도면 · 요소	근거 (경로:줄 · 기록)
E-18	GPT 호출 · 모델 · 대기 상한	`app/routers/copygen/llm.py:1-27, 120-141`
E-19	라운드 20 후보 · 최대 3라운드 · 최종 3개	`app/routers/copygen/pipeline.py:16-18` · `app/settings.py:143` (`candidate_n` 3)
E-20	코드 필터 사유	`app/routers/copygen/filters.py:23-34`
E-21	GraphJudge → `app.api.judge`	`app/routers/copygen/judge.py:112-136`
E-22	AI 광고 생성 편집기 판정	`app/routers/user.py:2109-2153` (`compose_judge`)
E-23	검수 그래프 · 종착 넷 · 루프 없음	`app/graph.py:4435-4447` (`build_review`) · `:4121-4164` (`REVIEW_TERMINALS` · `route_review`)
E-24	코어 노드 순서 · 법별 팬아웃	`app/graph.py:3997-4006, 4398-4418` · `:491-506` (`LAW_SCOPE` · `LAW_OF_NODE`)
E-25	`classify` 는 받은 품목만 씀	`app/graph.py:586-590`
E-26	`retrieve` → 조문 검색 · 두 갈래를 법마다 따로 받음(섞지 않음) · 법별 노드 · `references` 가 `law_view` 로 섞음(RRF k=60 · 규범당 2) · U2 허용분	`app/graph.py:668-708` (`retrieve` → `rt.wide`) · `:1335-1365` (`_law_node` → `rt.law_view`) · `:3928-3964` (`references`) · `app/retrieve.py:904-922` (`law_view`) · `:959-978` (`search`) · `:980-990` (`wide`) · `:1-40, 69, 253-301` · `app/settings.py:113, 117, 123` (폭 50 · `rrf_k` 60 · `per_law_cap` 2)
E-27	질의 임베딩 KURE-v1	`app/retrieve.py:391-410` · `app/settings.py:169-186`
E-28	`match_dict` → `dict_entry` (칸 C 단독판정 = 확정 재료 · 나머지 = 유형 후보)	`app/graph.py:716-722, 748-800` (`load_dict_entries` · `load_weak_entries` · `_viewed`)
E-29	`encode` → 판정 인코더 · `COPYLANE_MODEL_DIR` · 잠금	`app/graph.py:1124-1130` · `app/encoder.py:1-37, 122-131`
E-30	`assess_risk` → `v_risk_lookup`	`app/graph.py:1831-1840, 2090-2095` · `db/schema.sql:574-581`
E-31	`doc_rules` 의 인코더 쪽 판정 · 조건부 위반 · 품목 모를 때 기록 질문 걷기	`app/graph.py:2874-2955` (`doc_rules`) · `:2957-2971` (`_pick_first` · D-348 보충 37 결정 2) · `:3828-3852` (`_with_condition` → `_encoder_side`)
E-32	`references` 와 조문 원문(`document` · `v_current_chunk`)	`app/graph.py:3928-3933, 841-853, 943-944`
E-33	추적이 켜져 있으면 멈춤	`app/graph.py:4377-4391`
E-34	생성 그래프는 스텁	`app/graph.py:4450-4469`
E-35	데이터층 19표 · 런타임층 12표 · alembic head	`db/schema.sql` · `app/models.py:55-607` · `alembic/versions/20261016_0027_judgment_encoder_basis.py:22` · 현행 기준표 2-8
E-36	관리자 화면이 운영 표를 읽음	`app/routers/admin_dbread.py:1-12`
E-37	로컬 DB 컨테이너 · `127.0.0.1:5432`	`docker-compose.yml:19-29` · `app/settings.py:55`
E-38	모델 잠금	`models.lock.json:188-208` · `app/model_lock.py`
E-39	수집 게이트	`collect/registry.py:107-150`
E-40	수집기 목록 · 사람이 받는 원천	`collect/__init__.py:37-130` (`COLLECTORS` · `MANUAL_SOURCES`) · `collect/env.py:27-52`

번호	도면 · 요소	근거 (경로:줄 · 기록)
E-41	전처리 · 골든셋 · 청킹 · 적재 · 임베딩 명령	`launcher.py:1566-1615, 1672-1790` · `scripts/load_db.py:86-1086` · `scripts/embed.py:1-20`
E-42	파생물 공유 · 기기 역할	`scripts/data_store.py:1-25, 249, 442, 678-769`
E-43	런처 메뉴 번호	`launcher.py:2003-2084`
E-44	AWS g4dn · Ubuntu 24.04 · T4 · 배포 단계 · 포트 점검	`origin/ssm:docs/ssm/배포/deploy_server.sh:1-60, 81, 427-516`
E-45	systemd `copylane-app` · `copylane-sllm`	`origin/ssm:docs/ssm/배포/deploy_server.sh:470-533` · README 「지금 어디까지」(`cleanup-adhoc · services` 완료 표시)
E-46	Caddy 임시판 `:80` · 닫는 경로 · 1MB(저장소 판) · 서버 적용은 대기	`origin/ssm:docs/ssm/배포/Caddyfile.ip:1-38` · README 「지금 어디까지」(`Caddyfile.ip` 적용 표시가 미완) · `작업기록_1016.md:66` (지금 판 「닫는 경로 없음」)
E-47	서버 에디션 `cloud` / `demo` 만 허용 · `COPYLANE_SLLM_URL` 설정	`origin/ssm:docs/ssm/배포/deploy_server.sh:281-305, 371-374` · `작업기록_1019.md` (verify 칸 `EDITION=cloud`)
E-48	RDS PostgreSQL 16 · db.t4g.small · 파생물 139개 · 인코더 D4-20 · 임베딩 GPU · sLLM · 카피 생성	`origin/ssm:docs/ssm/배포/작업기록_1019.md` · README 「지금 어디까지 (2026-10-21)」
E-49	도메인 · HTTPS · 부하 시험 · 관리자 IP 제한 아직	`origin/ssm:docs/ssm/배포/README.md` 같은 표
E-50	배포 계획(10-06 판)의 구성	`docs/05_배포/배포계획.md:1-60`
E-51	모델 학습 위치(인코더 D4-20 Colab T4 · sLLM v14 RTX 3080 · 확장 판 RunPod RTX PRO 4500)	현행 기준표 4-1 · 4-4 · 11절 정정 · `docs/lse/자체_sLLM_결과서_제출본_2026-10-14.md:59` · `docs/psj/reports/D4_실험기록_20261012.md:1081-1122` · `docs/lse/자체_sLLM_결과서_제출본_2026-10-19.md:55-59`
E-52	다중 에이전트 = 법별 판정 노드 셋 (파이프라인형)	`docs/02_설계/AI_시스템_아키텍처.md:37-38`
E-53	응답의 보류 종류 계산 칸 `hold_kind`	`app/contracts.py:71-86` (`HoldKind`) · `:673-676` (`SentenceJudgment.hold_kind` · computed_field) · `:1203-1222` (`hold_kind`) · 결정기록 D-348 보충 36 결정 4
E-54	판정 응답 뒤에 붙는 두 겹 안내(쉬운 풀이 · 원문 보기 · 화면 미표시)	`app/graph.py:4349-4351` (`to_response` → `guide.attach`) · `app/guide.py:1-15` · 결정기록 D-348 보충 37 결정 7